

Mastery Workbook 5

Joe Ricks

March 1, 2026

I have neither given nor received unauthorized assistance.

#1 (10 pts)

Look up the RSA algorithm and write 3 specific questions or observations you have about how it works.

- 1.
- 2.
- 3.

#2 (30 pts)

Summarize essential definitions and concepts from Sec. 4.1 - 4.3.

#2 (continued)

#3 (10 pts)

Read “The Sieve of Eratosthenes” on page 259. Notice it uses THM 2, when it says “note that the composite integers not exceeding 100 must have a prime factor not exceeding 10.”

- A. Consider Theorem 2 page 258 - show inductively that it is true with some examples.
- B. Re-Prove Theorem 2 page 258 in your own words, using our proof structure for this proof by contradiction.

#4 (10 pts)

Recall that we can think of modulo in terms of scoops and cups. The Euclidean Algorithm can also apply to scoops and cups and solves the question: “Given 2 measuring scoops, of a cups and b cups, what is the smallest measure in cups that can be made?”

You must bake a cake and need to measure exactly 1 cup. Can you make the cake given:

a. 2 scoops measuring 17 and 88?

b. 2 scoops measuring 35 and 88?

c. 2 scoops measuring 37 and 53?

#5 (10 pts)

Following the Square and Mod video, create a “Square and Mod Chart” for powers of 5 mod 17.

Show some EVIDENCE OF THE PROCESS you are using to get the values below.

$$5^0 \bmod 17 =$$

$$5^1 \bmod 17 =$$

$$5^2 \bmod 17 =$$

$$5^4 \bmod 17 =$$

$$5^8 \bmod 17 =$$

$$5^{16} \bmod 17 =$$

$$5^{32} \bmod 17 =$$

$$5^{64} \bmod 17 =$$

$$5^{128} \bmod 17 =$$

$$5^{256} \bmod 17 =$$

#6 (10 pts)

Use the chart and the Square 'n Mod method from the video to find:

- $5^{270} \bmod 17$

- $5^{186} \bmod 17$

- $5^{411} \bmod 17$

#7 (10 pts)

Choose an FME algorithm to code:

- Book Version
- Sriram's Version
- Your own based on Square 'n Mod

Briefly, explain why you chose which algorithm you did.

Include a screenshot of your COMMENTED code below:

#8 (10 pts)

To motivate our use of Fast Modular Exponentiation FME, let's do a comparison of two ways we can calculate $a^n \bmod m$.

Write a new function called NOT_FME that returns the result of $a^n \bmod m$, using just the standard exponent calculation and the built in Python Mod function %.

Then use the Python function pow(), which calculates the same result using FME.

Do your own informal investigation to see if FME has a greater impact on the runtime when n is getting large or when a is getting large.

Summary of Results: